

Paquets de R per a la bioinformàtica

Programari per a l'anàlisi de dades

Robert Buj

18/02/2026

Taula de continguts

Exercicis i casos pràctics amb R	2
Exercici 1. Aplicació Shiny a partir d'un conjunt de dades	2
Exercici 2. Explorador de dades	3
Exercici 3. Aplicació Shiny per a conjunt de dades Iris	6
Exercici 4. Bioconductor amb el paquet GenomicFeatures	7
Exercici 5. Bioconductor amb el paquet limma	8
Exercici 6. Bioconductor amb ComplexHeatMap	9
Exercici 7. Bioconductor amb Biostrings	10
Exercici 8. Aplicacions del paquet caret	11
Exercici 9. Aplicacions del paquet caret. Model de classificació random forest	12
Exercici 10. Aplicació del paquet igraph a l'àmbit de les proteïnes	13
Cas pràctic 1	14

Exercicis i casos pràctics amb R

Exercici 1. Aplicació Shiny a partir d'un conjunt de dades

A partir del conjunt de dades `heart` del paquet `survival`, feu una aplicació Shiny que a partir de la selecció de dues variables del conjunt de dades representi un gràfic de dispersió i un de línies, a més d'un histograma sol per a la variable horitzontal seleccionada. Per fer aquest exercici podeu basar-vos en l'Exemple 2 realitzat en l'apartat 1.1. Ecosistema de R: Entorns reproduïbles i visualització de dades. S'aconsella ampliar coneixements en l'ús de diferents paràmetres en les funcions definides, així com explorar un altre tipus de funcionalitats per aprofundir en l'estudi del conjunt de dades, a criteri de l'estudiant.

Resposta:

No executeu el codi de sota, ja que és una aplicació Shiny que s'ha de executar des del terminal.

```
conda activate ./conda
Rscript LAB6/ex1.R
```

Codi de l'aplicació Shiny:

```
1  # Carreguem els paquets necessaris
2  library(shiny)
3  library(survival)
4  library(ggplot2)
5  # Carreguem el conjunt de dades "heart"
6  data("heart")
7  # Definim la interfície de l'aplicació Shiny
8  ui <- fluidPage(
9    titlePanel("Conjunt de dades heart"),
10    sidebarLayout(
11      sidebarPanel(
12        selectInput(
13          "x", "Seleccioneu la variable X:",
14          choices = colnames(heart)
15        ),
16        selectInput(
17          "y", "Seleccioneu la variable Y:",
18          choices = colnames(heart)
19        ),
20        selectInput(
21          "plot_type", "Seleccioneu el tipus de gràfic:",
22          choices = c(
23            "Gràfic de dispersió" = "scatter",
24            "Línies" = "lines",
25            "Histograma" = "histogram"
26          )
27        ),
28        hr()
29      ),
30      mainPanel(
31        # Dibuixem el gràfic seleccionat
32        plotOutput("plot")
33      )
34    )
35 )
36 # Definim la lògica de l'aplicació Shiny
37 server <- function(input, output) {
```

```

38   # Creem el gràfic
39   output$plot <- renderPlot({
40     x <- input$x
41     y <- input$y
42     plot_type <- input$plot_type
43     if (plot_type == "scatter") {
44       ggplot(heart, aes(x = .data[[x]], y = .data[[y]])) +
45         geom_point() +
46         xlab(x) +
47         ylab(y) +
48         ggtitle("Gràfic de Dispersió")
49     } else if (plot_type == "lines") {
50       ggplot(heart, aes(x = .data[[x]], y = .data[[y]], group = 1)) +
51         geom_line() +
52         xlab(x) +
53         ylab(y) +
54         ggtitle("Gràfic de Línies")
55     } else {
56       if (is.numeric(heart[[x]])) {
57         ggplot(heart, aes(x = .data[[x]])) +
58           geom_histogram(fill = "blue", color = "black", bins = 30) +
59           xlab(x) +
60           ylab("Freqüència") +
61           ggtitle("Histograma")
62       } else {
63         ggplot(heart, aes(x = .data[[x]])) +
64           geom_bar(fill = "blue", color = "black") +
65           xlab(x) +
66           ylab("Freqüència") +
67           ggtitle("Gràfic de Barres (variable categòrica)")
68       }
69     }
70   })
71 }
72 # Executem l'aplicació Shiny
73 shinyApp(
74   ui = ui,
75   server = server,
76   options = list(
77     launch.browser = TRUE
78   )
79 )

```

Exercici 2. Explorador de dades

Mostreu un conjunt de dades a partir d'un arxiu .csv a la vostra elecció que mostri també el resum estadístic del conjunt de dades triat. D'altra banda, afegiu la selecció d'una variable del conjunt de dades a partir de la qual es representi un gràfic del tipus histograma.

Resposta:

No executeu el codi de sota, ja que és una aplicació Shiny que s'ha de executar des del terminal.

```

conda activate ./conda
Rscript LAB6/ex2.R

```

Codi de l'aplicació Shiny:

```
1  # Carreguem els paquets necessaris
2  library(shiny)
3  library(ggplot2)
4  library(dplyr)
5  # Carreguem el conjunt de dades per defecte
6  if (file.exists("data.csv")) {
7    dataset <- read.csv("data.csv")
8  } else if (file.exists("LAB6/data.csv")) {
9    dataset <- read.csv("LAB6/data.csv")
10 } else {
11   stop("No s'ha trobat el fitxer de dades.")
12 }
13 # Definim la interfície de l'aplicació shiny
14 ui <- fluidPage(
15   titlePanel("Explorador de conjunt de dades"),
16   sidebarLayout(
17     sidebarPanel(
18       selectInput("x", "Seleccioneu la variable X:",
19         choices = names(dataset), selected = names(dataset)[1]
20     ),
21     selectInput("y", "Seleccioneu la variable Y:",
22       choices = names(dataset), selected = names(dataset)[2]
23     ),
24     selectInput(
25       "plot_type",
26       "Seleccioneu el tipus de gràfic:",
27       c("scatterplot", "barplot", "boxplot", "histogram")
28     ),
29     actionButton("show_summary", "Resum estadístic")
30   ),
31   mainPanel(
32     uiOutput("compatibility_box"),
33     plotOutput("plot"),
34     verbatimTextOutput("summary_text")
35   )
36 )
37 )
38 # Definim la lògica de l'aplicació shiny
39 server <- function(input, output, session) {
40   data <- reactive({
41     dataset
42   })
43
44   compatibility_reason <- reactive({
45     req(data(), input$x, input$plot_type)
46
47     x_is_numeric <- is.numeric(data()[[input$x]])
48     y_is_numeric <- if (!is.null(input$y) && nzchar(input$y)) {
49       is.numeric(data()[[input$y]])
50     } else {
51       FALSE
52     }
53   })
54 }
```

```

53
54   if (input$plot_type == "scatterplot" && (!x_is_numeric || !y_is_numeric)) {
55     return("El scatterplot requereix variables numèriques tant a X com a Y.")
56   }
57
58   if (input$plot_type == "histogram" && !x_is_numeric) {
59     return("L'histograma requereix una variable numèrica a X.")
60   }
61
62   if (input$plot_type == "boxplot" && !y_is_numeric) {
63     return("El boxplot requereix una variable numèrica a Y.")
64   }
65
66   NULL
67 })
68
69 output$compatibility_box <- renderUI({
70   reason <- compatibility_reason()
71   if (is.null(reason)) {
72     return(NULL)
73   }
74
75   div(
76     style = "border: 1px solid #f1c40f; background-color: #fff8e1; padding: 12px; border-radius:
77     strong("Incompatibilitat de variables per al tipus de gràfic seleccionat."),
78     p(reason, style = "margin: 8px 0 4px 0;"),
79     p("Per a més informació, consulteu el resum estadístic de les variables.", style = "margin:
80   )
81 })
82
83 output$plot <- renderPlot({
84   req(data())
85   validate(need(is.null(compatibility_reason()), compatibility_reason()))
86
87   if (input$plot_type == "scatterplot") {
88     ggplot(data(), aes(x = .data[[input$x]], y = .data[[input$y]])) +
89     geom_point()
90   } else if (input$plot_type == "barplot") {
91     ggplot(data(), aes(x = .data[[input$x]])) +
92     geom_bar()
93   } else if (input$plot_type == "boxplot") {
94     ggplot(data(), aes(x = .data[[input$x]], y = .data[[input$y]])) +
95     geom_boxplot()
96   } else if (input$plot_type == "histogram") {
97     ggplot(data(), aes(x = .data[[input$x]])) +
98     geom_histogram()
99   }
100 })
101 output$summary_text <- renderPrint({
102   req(data())
103   if (input$show_summary > 0) {
104     summary(data())
105   }
106 })

```

```

107 }
108 # Executem l'aplicació shiny
109 shinyApp(
110   ui = ui,
111   server = server,
112   options = list(
113     launch.browser = TRUE
114   )
115 )

```

Exercici 3. Aplicació Shiny per a conjunt de dades Iris

A partir del conjunt de dades *Iris*, feu una aplicació Shiny per mostrar, a partir d'una de les variables per a seleccionar del conjunt de dades, els gràfics *boxplot* i *barplot*.

Resposta:

No executeu el codi de sota, ja que és una aplicació Shiny que s'ha de executar des del terminal.

```

conda activate ./conda
Rscript LAB6/ex3.R

```

Codi de l'aplicació Shiny:

```

1  # Carreguem les biblioteques necessàries
2  library(shiny)
3  library(ggplot2)
4  # Carreguem un conjunt de dades d'exemple (en aquest cas, Iris)
5  data("iris")
6  # Definim la interfície de l'aplicació Shiny
7  ui <- fluidPage(
8    titlePanel("Explorador de Dades amb Gràfics"),
9    sidebarLayout(
10     sidebarPanel(
11       # Selector de variables x
12       selectInput(
13         "x",
14         "Seleccioneu la variable X:",
15         choices = colnames(iris)
16       ),
17       # Selector de tipus de gràfic
18       selectInput(
19         "plot_type",
20         "Seleccioneu el tipus de gràfic:",
21         choices = c("Bar Plot", "Box Plot")
22       )
23     ),
24     mainPanel(
25       # Gràfic resultant
26       plotOutput("output_plot")
27     )
28   )
29 )
30 # Definim la lògica de l'aplicació Shiny
31 server <- function(input, output) {
32   # Creeu el gràfic seleccionat

```

```

33 output$output_plot <- renderPlot({
34   x <- input$x
35   data <- iris
36   if (input$plot_type == "Bar Plot") {
37     ggplot(data, aes(x = .data[[x]])) +
38       geom_bar() +
39       xlab(x) +
40       ylab("Frequency")
41   } else if (input$plot_type == "Box Plot") {
42     ggplot(data, aes(x = .data[[x]], y = .data[["Sepal.Length"]])) +
43       geom_boxplot() +
44       xlab(x) +
45       ylab("Sepal Length")
46   }
47 })
48 }
49 # Executem l'aplicació Shiny
50 shinyApp(
51   ui = ui,
52   server = server,
53   options = list(
54     launch.browser = TRUE
55   )
56 )

```

Exercici 4. Bioconductor amb el paquet GenomicFeatures

Utilitzant el paquet `GenomicFeatures`, definiu un vector amb la informació de cada gen, la posició inicial i final, la direcció del bri (`strand`), l'identificador del gen (`id`) i el tipus de gen (`type`).

Una orientació per definir algunes d'aquestes dades les podeu trobar a l'Exemple 4. *Bioconductor utilitzant el paquet GenomicFeatures*, i podem definir la direcció del bri i el tipus de gen de la manera següent:

```
strand = c("+", "-", "+"), gene_type = c("protein_coding", "non_coding", "protein_coding")
```

Mostreu els detalls utilitzant un objecte de tipus `GRanges`.

Resposta:

```

1  # Carreguem el paquet GenomicFeatures
2  library(GenomicFeatures)
3  # Creem un conjunt de gens més detallat
4  gens_detalls <- data.frame(
5    seqnames = c("chr1", "chr2", "chr1"),
6    start = c(100, 200, 300),
7    end = c(200, 300, 400),
8    gene_id = c("Gene1", "Gene2", "Gene3"),
9    strand = c("+", "-", "+"),
10   gene_type = c("protein_coding", "non_coding", "protein_coding"),
11   stringsAsFactors = FALSE
12 )
13 # Creem un objecte `GRanges` que representa la informació dels gens
14 gr_detalls <- GRanges(
15   seqnames = gens_detalls$seqnames,
16   ranges = IRanges(

```

```

17     start = gens_detalls$start, end =
18         gens_detalls$end
19 ),
20     strand = gens_detalls$strand,
21     id = gens_detalls$gene_id,
22     type = gens_detalls$gene_type
23 )
24 # Mostrem la informació detallada dels gens
25 print(gr_detalls)

## GRanges object with 3 ranges and 2 metadata columns:
##      seqnames      ranges strand |          id          type
##      <Rle> <IRanges> <Rle> | <character>    <character>
## [1]   chr1    100-200      + |   Gene1 protein_coding
## [2]   chr2    200-300      - |   Gene2  non_coding
## [3]   chr1    300-400      + |   Gene3 protein_coding
## -----
##      seqinfo: 2 sequences from an unspecified genome; no seqlengths

```

Exercici 5. Bioconductor amb el paquet limma

El paquet *limma* és un paquet d'R que s'utilitza per analitzar dades d'expressió gènica i fer anàlisis estadístiques en experiments de bioxips i seqüenciació d'ARN (ARN-Seq). La sigla *limma* significa *linear models for biochip data*. A partir de les dades següents:

Gen	Sample1	Sample2	Sample3	Sample4	Sample5	Sample6
Gene1	5	6	7	8	9	10
Gene2	4	5	6	7	8	9
Gene3	3	4	5	6	7	8

Feu una anàlisi d'expressió gènica diferencial amb el paquet *limma*.

Resposta:

```

1  # Carreguem el paquet necessari
2  library(limma)
3  # Carreguem dades de mostra
4  data <- read.table(header = TRUE, text = "
5  Gene Sample1 Sample2 Sample3 Sample4 Sample5 Sample6
6  Gene1 5 6 7 8 9 10
7  Gene2 4 5 6 7 8 9
8  Gene3 3 4 5 6 7 8
9  ")
10 # Definim els grups
11 group <- factor(c("Group1", "Group1", "Group1", "Group2", "Group2", "Group2"))
12 # Creem un objecte DGEList
13 exprs_data <- data[, -1]
14 rownames(exprs_data) <- data$Gene
15 group <- factor(group)
16 design <- model.matrix(~ 0 + group)
17 colnames(design) <- levels(group)
18 fit <- lmFit(exprs_data, design)
19 # Fem l'anàlisi d'expressió gènica diferencial amb limma

```



```

20 contrast <- makeContrasts(Group2 - Group1, levels = design)
21 fit2 <- contrasts.fit(fit, contrast)
22 fit2 <- eBayes(fit2)
23 results <- topTable(fit2, coef = 1)
24 # Mostrem els resultats
25 print(results)

```

```

##      logFC AveExpr      t      P.Value  adj.P.Val      B
## Gene1      3      7.5 3.674235 0.003182214 0.003182214 0.2754422
## Gene2      3      6.5 3.674235 0.003182214 0.003182214 0.2754422
## Gene3      3      5.5 3.674235 0.003182214 0.003182214 0.2754422

```

Exercici 6. Bioconductor amb ComplexHeatMap

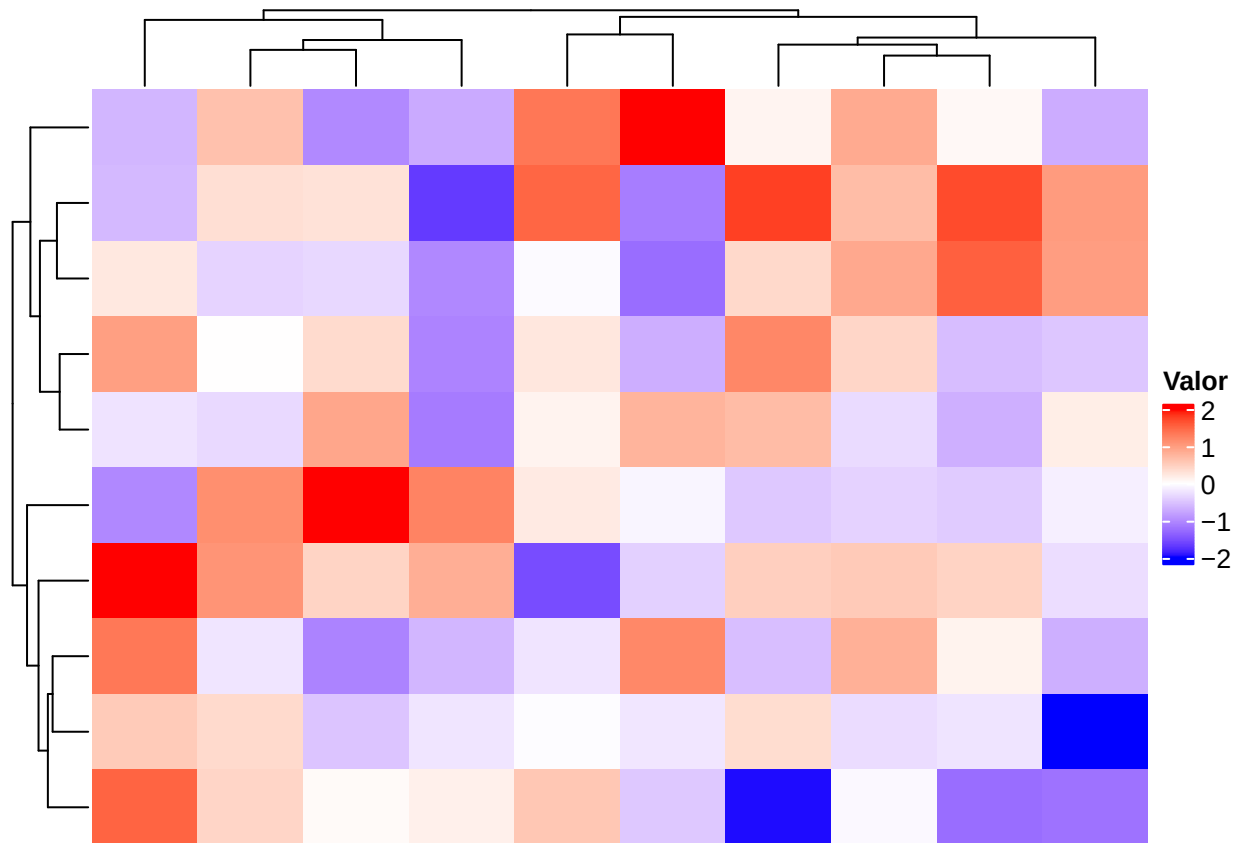
El paquet `ComplexHeatMap` és útil per visualitzar dades de matriu en forma de mapes de calor complexos amb anotacions addicionals. A partir d'una matriu fictícia, creeu un objecte `HeatMap`, mostreu-lo i interpreteu-ne els resultats.

Resposta:

```

1 library(ComplexHeatmap)
2 library(circlize)
3 # Simulem les dades d'una matriu a partir d'una de normal
4 set.seed(123)
5 matriu_exemple <- matrix(rnorm(100), ncol = 10)
6 # Creem un objecte Heatmap
7 heatmap_objecte <- Heatmap(matriu_exemple,
8   name = "Valor",
9   show_row_names = FALSE,
10  show_column_names = FALSE,
11  col = colorRamp2(c(-2, 0, 2), c(
12    "blue",
13    "white", "red"
14  ))
15 )
16 # Dibuixem el mapa de calor
17 draw(heatmap_objecte, heatmap_legend_side = "right")

```



El mapa de calor té l'objectiu de visualitzar patrons i tendències en les dades.

En aquest mapa de calor, els colors tenen la següent interpretació: - Blau: Representa valors baixos. - Blanc: Representa valors intermedis. - Vermell: Representa valors alts.

La intensitat del color indica el nivell d'expressió o la magnitud del valor associat. Colors més intensos representen valors més alts o més baixos, segons la paleta de colors utilitzada. Les columnes representen diferents mostres o condicions i les files representen diferents elements o gens. Els patrons horitzontals (en files) poden indicar similituds o diferències en l'expressió de gens entre diferents elements. Els patrons verticals (en columnes) poden indicar similituds o diferències en l'expressió d'un gen al llarg de diferents condicions. L'agrupació (clusterització) de files o columnes pot suggerir similituds en els perfils d'expressió gènica.

En aquest cas, les dades són simulades; no obstant això, en el context d'un estudi de gens, un canvi de color podria significar una resposta a determinades condicions.

Exercici 7. Bioconductor amb Biostrings

El paquet **Biostrings** de Bioconductor en R està dissenyat per a la manipulació i l'anàlisi de seqüències biològiques. Suposem que definim les seqüències següents:

```
sequencia1: "ATCGAATCGA" sequencia2: "TAGCTAGCTA"
```

Combineu les seqüències i obtingueu el complement invers de cada seqüència.

Resposta:

```
1 library(Biostrings)
2 # Creem les seqüències d'ADN
3 sequencia1 <- DNASTring("ATCGAATCGA")
4 sequencia2 <- DNASTring("TAGCTAGCTA")
```

```

5  # Combinem les seqüències en un DNASTringSet
6  sequencies <- DNASTringSet(c(sequencia1, sequencia2))
7  # Obtenim el complement invers de cada seqüència
8  comp_invers <- reverseComplement(sequencies)
9  # Imprimim les seqüències originals i els seus complements inversos
10 cat("Sequencies Originals:\n")

## Sequencies Originals:
1  print(sequencies)

## DNASTringSet object of length 1:
##      width seq
## [1]      20 ATCGAATCGATAGCTAGCTA

1  cat("\nComplements Inversos de les Sequencies:\n")

##
## Complements Inversos de les Sequencies:
1  print(comp_invers)

## DNASTringSet object of length 1:
##      width seq
## [1]      20 TAGCTAGCTATCGATTTCGAT

```

Exercici 8. Aplicacions del paquet caret

A partir del paquet `caret` i del conjunt de dades Iris, creeu una partició del conjunt de dades sobre la variable `species` i creeu i entreneu un model de k-NN per fer prediccions. Finalment, mostreu la matriu de confusió i comenteu el resultat.

Resposta:

```

1  # Carreguem el paquet necessari
2  library(caret)
3  # Carreguem el conjunt de dades iris
4  data(iris)
5  # Creem una partició del conjunt de dades
6  set.seed(123)
7  training_indices <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
8  training_data <- iris[training_indices, ]
9  testing_data <- iris[-training_indices, ]
10 # Creem i entrenem un model de k-NN amb caret
11 model <- train(
12   Species ~ .,
13   data = training_data, method = "knn",
14   trControl = trainControl(method = "cv")
15 )
16 # Fem prediccions en el conjunt de prova
17 predictions <- predict(model, testing_data)
18 # Mostrem la matriu de confusió i altres mètriques
19 conf_matrix <- confusionMatrix(predictions, testing_data$Species)
20 print(conf_matrix)

## Confusion Matrix and Statistics
##

```

```

##               Reference
## Prediction   setosa versicolor virginica
##   setosa      15         0         0
##   versicolor   0        15         1
##   virginica    0         0        14
##
## Overall Statistics
##
##               Accuracy : 0.9778
##               95% CI   : (0.8823, 0.9994)
##   No Information Rate : 0.3333
##   P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa   : 0.9667
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##               Class: setosa Class: versicolor Class: virginica
## Sensitivity      1.0000      1.0000      0.9333
## Specificity      1.0000      0.9667      1.0000
## Pos Pred Value   1.0000      0.9375      1.0000
## Neg Pred Value   1.0000      1.0000      0.9677
## Prevalence       0.3333      0.3333      0.3333
## Detection Rate   0.3333      0.3333      0.3111
## Detection Prevalence 0.3333      0.3556      0.3111
## Balanced Accuracy 1.0000      0.9833      0.9667

```

Segons el valor d'Accuracy, la precisió del model és del 97.78 %, la qual cosa indica que el 97.78 % de les prediccions van ser correctes en el conjunt de prova. Com que el valor de kappa és de 0.9667, és possible que hi hagi concordança significativa entre les prediccions del model i les classes d'espècies reals.

Setosa: El model té un rendiment perfecte per a la classe Setosa, amb sensibilitat, especificitat i valor predictiu positiu del 100 %.

Versicolor: També té un rendiment sòlid per a Versicolor, amb sensibilitat i valor predictiu positiu del 100 %, encara que l'especificitat és més baixa.

Virginica: El model té una sensibilitat del 93.33 % per a la classe Virginica.

El Balanced Accuracy indica que el model mostra un bon equilibri en la precisió de les tres classes, amb una precisió equilibrada del 100 % per a Setosa, 98.33 % per a Versicolor i 96.67 % per a Virginica.

Així doncs, el model de k-NN sembla bastant precís en la classificació de les flors en el conjunt de dades Iris, amb un rendiment més fort per a les classes Setosa i Versicolor.

Exercici 9. Aplicacions del paquet caret. Model de classificació random forest

Suposem que tenim les següents dades de 5 pacients als quals se'ls han practicat algunes proves mèdiques i on, entre altres dades, s'indica si tenen o no la malaltia estudiada:

```

Edat = 30, 45, 50, 22, 65 Sexe = "M", "F", "M", "F", "M"
ResultatProva1 = 0.1, 0.5, 0.8, 1.2, 0.9
ResultatProva2 = 12, 9, 15, 8, 10
Malaltia = "No", "No", "Sí", "Sí", "Sí"

```

Es demana entrenar un model de classificació supervisada, per exemple, el *random forest*.

Resposta:

```
1 library(caret)
2 # Creem un dataframe d'exemple
3 dades <- data.frame(
4   Edat = c(30, 45, 50, 22, 65),
5   Sexe = c("M", "F", "M", "F", "M"),
6   ResultatProva1 = c(0.1, 0.5, 0.8, 1.2, 0.9),
7   ResultatProva2 = c(12, 9, 15, 8, 10),
8   Malaltia = c("No", "No", "Sí", "Sí", "Sí")
9 )
10 # Convertim la variable Sexe en factor
11 dades$Sexe <- as.factor(dades$Sexe)
12 # Definim el control de l'entrenament, validació creuada amb 5 folds
13 control <- trainControl(method = "cv", number = 5)
14 # Entrenem un model de classificació (per exemple, un model de bosc aleatori)
15 model <- train(Malaltia ~ ., data = dades, method = "rf", trControl = control)
16 # Vegem els resultats del model
17 print(model)
```

```
## Random Forest
##
## 5 samples
## 4 predictors
## 2 classes: 'No', 'Sí'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 4, 4, 4, 4, 4
## Resampling results across tuning parameters:
##
##  mtry  Accuracy  Kappa
##  2     0.4      0
##  3     0.6      0
##  4     0.6      0
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 3.
```

La mostra és massa petita per extreure conclusions, no obstant això, de les dades obtingudes es pot suggerir que el model aconsegueix una precisió màxima del 60 %.

Exercici 10. Aplicació del paquet igraph a l'àmbit de les proteïnes

Suposem que tenim la següent matriu d'adjacència que representa les interaccions entre proteïnes:

```
matrix_adjacencia ( 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0 )
```

Es demana representar gràficament la xarxa d'interaccions entre aquestes proteïnes.

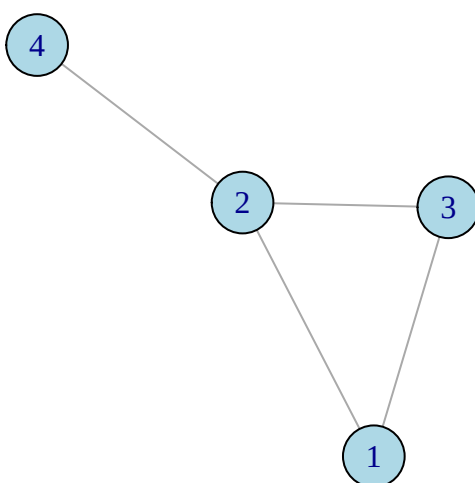
Resposta:

```
1 # Creem la matriu d'adjacència que representa interaccions entre proteïnes
2 matrix_adjacencia <- matrix(c(
3   0, 1, 1, 0,
4   1, 0, 1, 1,
5   1, 1, 0, 0,
```

```

6     0, 1, 0, 0
7   ), nrow = 4, byrow = TRUE)
8   # Convertim la matriu en un objecte graph d'igraph
9   xarxa_proteines <- igraph::graph_from_adjacency_matrix(
10     matriu_adjacencia,
11     mode = "undirected",
12     diag = FALSE
13   )
14   # Visualitzem la xarxa d'interaccions entre proteïnes
15   plot(xarxa_proteines,
16     layout = igraph::layout_with_fr, # Algorisme de disposició per organitzar els nodes
17     vertex.label = 1:4, # Etiquetes dels nodes
18     vertex.color = "lightblue", # Color dels nodes
19     vertex.size = 30 # Grandària dels nodes
20   )

```



Cas pràctic 1

A partir de Bioconductor, i utilitzant el paquet **Biostrings**, es demana fer una aplicació senzilla amb Shiny que sigui un comptador d'ocurrències de patrons d'ADN.

Es requereix que la interfície que s'obtingui sigui similar a la següent:

Per fer proves, es poden fer servir els valors següents:

seqüència: ATCGATCGATCGATCGATCGATCGATCGATCG

patró: ATCG

Resposta:

No executeu el codi de sota, ja que és una aplicació Shiny que s'ha de executar des del terminal.

```

conda activate ./conda
Rscript LAB6/cp1.R

```

Codi de l'aplicació Shiny:

```

1 library(shiny)
2 library(Biostrings)
3 # Definim la interfície de l'aplicació shiny
4 ui <- fluidPage(
5   titlePanel("Comptador d'Ocurrències de Patrons d'ADN"),

```

```

6     sidebarLayout(
7       sidebarPanel(
8         textInput("secuencia", "Introduïu una seqüència d'ADN:"),
9         textInput("patron", "Introduïu el patró a buscar:"),
10        actionButton("comptador", "Comptar ocurrencies"),
11        br(),
12        verbatimTextOutput("resultat")
13      ),
14      mainPanel()
15    )
16  )
17  # Definim la lògica de l'aplicació Shiny
18  server <- function(input, output) {
19    output$resultat <- renderPrint({
20      req(input$comptador)
21      isolate({
22        # Obtenim la seqüència d'ADN introduïda
23        adn_sequencia <- DNASString(input$secuencia)
24        # Obtenim el patró introduït
25        patron <- DNASString(input$patron)
26        # Comptem les ocurrencies del patró a la seqüència
27        comptador <- countPattern(patron, adn_sequencia)
28        # Retornem el resultat
29        paste("Nombre d'ocurrencies:", comptador)
30      })
31    })
32  }
33  # Executem l'aplicació Shiny
34  shinyApp(
35    ui = ui,
36    server = server,
37    options = list(
38      launch.browser = TRUE
39    )
40  )

```